

GeoVisor Municipal - Documentación Técnica Completa

Municipio: Santander de Quilichao, Cauca, Colombia Versión: 1.0.0 Fecha de última actualización: 2026-03-31

Tabla de Contenidos

- 1. [Descripción General](#)
- 2. [Stack Tecnológico](#)
- 3. [Arquitectura del Sistema](#)
- 4. [Estructura de Carpetas](#)
- 5. [Frontend - React + OpenLayers](#)
- 6. [Backend - Node.js + Express](#)
- 7. [GeoServer - Capas WMS/WFS](#)
- 8. [Autenticación y Roles](#)
- 9. [Flujos Principales](#)
- 10. [Herramientas SIG](#)
- 11. [Panel de Estadísticas](#)
- 12. [Configuración y Despliegue](#)
- 13. [Usuarios de Desarrollo](#)
- 14. [Estado Actual del Proyecto](#)

1. Descripción General

El **GeoVisor Municipal** es una aplicación web de información geográfica (SIG) desarrollada para la Alcaldía Municipal de Santander de Quilichao. Permite visualizar, consultar y analizar capas de información territorial del municipio, organizadas por secretarías.

Capacidades principales

- Visualización de más de 25 capas geográficas sobre un mapa base de OpenStreetMap
- Consulta de atributos por clic en el mapa (GetFeatureInfo)
- Herramientas de medición de distancias y áreas
- Generación de buffers (zonas de influencia)
- Selección de entidades por polígono
- Estadísticas dinámicas por Unidad Barrial de Atención (UBA) y uso de suelo
- Control de acceso por secretaría mediante roles y JWT
- Panel de capas jerárquico organizado por dependencias municipales

2. Stack Tecnológico

Frontend

Tecnología	Versión	Uso
React	18.2.0	Framework UI
Vite	5.1.0	Bundler y servidor de desarrollo

OpenLayers	9.0.0	Motor de mapas interactivos
React Router DOM	6.22.0	Navegación SPA
Axios	1.6.7	Cliente HTTP con interceptores JWT

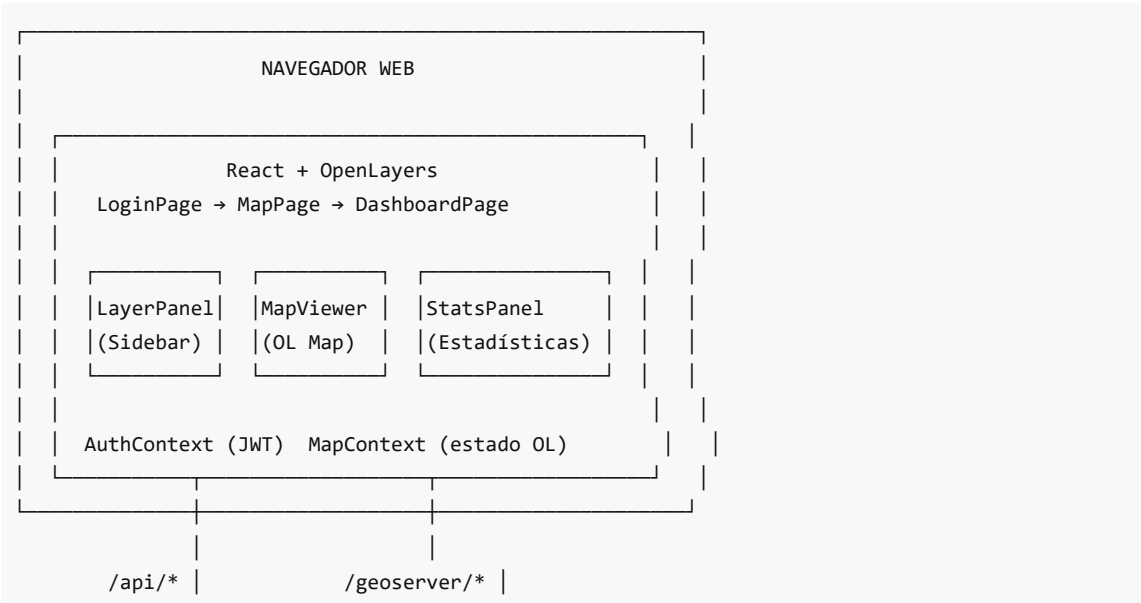
Backend

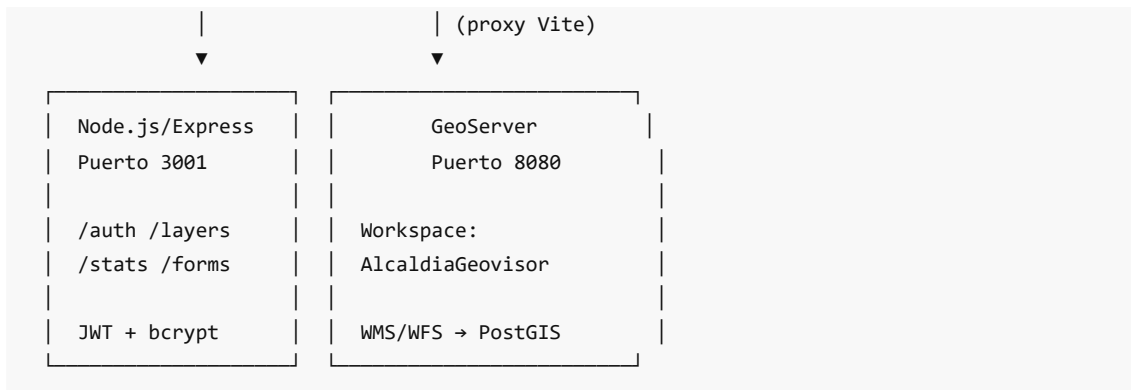
Tecnología	Versión	Uso
Node.js	>= 18	Runtime
Express	4.18.2	Framework web
jsonwebtoken	9.0.2	Autenticación JWT
bcryptjs	2.4.3	Hash de contraseñas
cors	2.8.5	Control de origen cruzado
express-rate-limit	7.1.5	Límite de peticiones
dotenv	16.3.1	Variables de entorno

Infraestructura

Servicio	Puerto	Descripción
GeoServer	8080	Servidor de capas geoespaciales WMS/WFS
Backend API	3001	API REST Node.js
Frontend Vite	5173	Servidor de desarrollo React

3. Arquitectura del Sistema





Proxy de Vite

Vite actúa como proxy en desarrollo para evitar problemas de CORS:

```

/geoserver/* → http://localhost:8080/geoserver/*
/api/*      → http://localhost:3001/api/*
  
```

4. Estructura de Carpetas

```

App-Alcaldia/
├── alcaldia-geovisor/
│   ├── frontend/
│   │   ├── public/
│   │   │   └── logos/
│   │   │       ├── logocolombia.png
│   │   │       └── alcaldia.png
│   │   ├── src/
│   │   │   ├── config/
│   │   │   │   ├── geoserver.js    # URL, workspace, proyecciones, centro del mapa
│   │   │   │   ├── layers.js       # Definición completa de ~50 capas WMS/WFS
│   │   │   │   ├── roles.js        # Permisos por rol
│   │   │   │   └── secretarias.js   # Listado de las 14 secretarías
│   │   │   ├── context/
│   │   │   │   ├── AuthContext.jsx  # Estado de autenticación JWT global
│   │   │   │   └── MapContext.jsx    # Estado del mapa OpenLayers global
│   │   │   ├── hooks/
│   │   │   │   └── useMapStats.js    # Hook para conteos dinámicos por UBA
│   │   │   ├── services/
│   │   │   │   ├── api.js           # Cliente axios configurado con interceptores
│   │   │   │   └── statsService.js   # Consultas WFS para estadísticas
│   │   │   └── organisms/
│   │   │       ├── Header/          # Navbar con logos y navegación
│   │   │       ├── MapViewer/       # Mapa principal OpenLayers (componente central)
│   │   │       ├── LayerPanel/      # Panel de capas con árbol jerárquico
│   │   │       ├── MapToolbar/      # Herramientas SIG (medir, buffer, seleccionar)
│   │   │       ├── StatsPanel/      # Estadísticas de UBAs y uso de suelo
│   │   │       ├── SearchPanel/     # Búsqueda de predios y barrios
│   │   │       └── SelectionResults/ # Resultados de selección por polígono
  
```

```

|   |   |   └─ molecules/
|   |   |       └─ UbaStats/           # Componente de estadísticas por UBA
|   |   |           └─ UsoSueloStats/  # Componente de estadísticas de uso de suelo
|   |   |   └─ pages/
|   |   |       └─ LoginPage.jsx        # Formulario de login
|   |   |       └─ MapPage.jsx          # Página principal del visor
|   |   |           └─ DashboardPage.jsx # Panel de accesos rápidos
|   |   |   └─ templates/
|   |   |       └─ MainLayout.jsx       # Layout contenedor con Outlet de React Router
|   |   |   └─ main.jsx                 # Entry point: providers y BrowserRouter
|   |   |   └─ App.jsx                  # Definición de rutas y PrivateRoute
|   |   |       └─ App.css               # Variables CSS globales, reset, estilos OL
|   └─ index.html
|   └─ package.json
|   └─ vite.config.js
└─ backend/
    └─ src/
        └─ routes/
            └─ auth.js                  # Login, logout, me, refresh
            └─ layers.js                 # Listado de capas por rol/secretaría
            └─ stats.js                  # Estadísticas de UBAs y uso de suelo
            └─ forms.js                  # Envío e historial de formularios
        └─ middleware/
            └─ authMiddleware.js         # authMiddleware, roleMiddleware, secretariaMiddleware
        └─ server.js                    # App Express con CORS, rate-limit, rutas
    └─ package.json

```

5. Frontend - React + OpenLayers

5.1 Punto de Entrada (main.jsx)

La aplicación se monta con la siguiente jerarquía de providers:

```

<BrowserRouter>
  <AuthProvider>           // Estado de autenticación JWT
    <MapProvider>           // Estado del mapa OpenLayers
      <App />               // Rutas y páginas
    </MapProvider>
  </AuthProvider>
</BrowserRouter>

```

5.2 Rutas (App.jsx)

Ruta	Componente	Protegida	Descripción
/login	LoginPage	No	Formulario de acceso
/mapa	MapPage	Sí	Visor geográfico principal
/dashboard	DashboardPage	Sí	Panel de accesos rápidos

/	—	—	Redirige a /mapa
*	—	—	Redirige a /mapa

PrivateRoute: Verifica `isAuthenticated` en `AuthContext`. Si no autenticado, redirige a `/login`.

5.3 AuthContext (context/AuthContext.jsx)

Gestiona el estado global de autenticación:

```
// Estado expuesto
user          // { id, username, name, role, secretaria }
token         // string JWT o null
isAuthenticated // boolean
loading       // boolean (durante verificación inicial)

// Métodos
login(username, password) // POST /api/auth/login → guarda token en localStorage
logout()                  // Limpia token y user
hasRole(role)             // boolean
hasPermission(permission) // boolean según tabla de permisos por rol
```

Tabla de permisos:

Rol	Permisos
admin	read, write, delete, manage_users, view_all
secretaria	read, write, view_own
lector	read, view_own
editor_geo	read, write, delete, manage_layers, view_all

Verificación al iniciar: Si hay token en `localStorage`, valida contra `GET /api/auth/me`. Si la respuesta es 401, elimina el token.

5.4 MapContext (context/MapContext.jsx)

Gestiona el estado global del mapa OpenLayers:

```
// Estado expuesto
map          // Instancia de ol/Map
mapReady     // boolean
activeLayers // Set<string> de IDs de capas visibles
selectedFeature // Último feature clickeado
activeTool   // 'measure-distance' | 'measure-area' | 'buffer' | 'select-polygon' | null
bufferRadius // número en metros (por defecto 100)
selectionResults // Array de features seleccionados por polígono

// Métodos
```

```

initMap(targetElement)          // Inicializa mapa en el div
addLayer(olLayer)                // Agrega capa al mapa
removeLayer(olLayer)            // Remueve capa del mapa
getLayerByName(name)            // Busca capa por propiedad 'name'
toggleLayerVisibility(layerName)
zoomToExtent(extent)
getFeatureInfoAt(coordinate, layers) // GetFeatureInfo WMS
clearTools()

```

Configuración del mapa:

- Centro: [-76.483765, 3.012569] (Santander de Quilichao)
- Zoom inicial: 14
- Proyección del mapa: EPSG:3857 (Web Mercator)
- Proyección de datos: EPSG:4326 (WGS84)
- Capa base: OpenStreetMap

5.5 MapViewer (organisms/MapViewer/MapViewer.jsx)

Componente central del visor. 687 líneas. Gestiona toda la lógica del mapa OpenLayers.

Responsabilidades principales:

Creación de capas

```

createWmsLayer(layerConfig)
// Crea una capa ImageWMS de OpenLayers
// URL: /geoserver/AlcaldiaGeovisor/wms
// Parámetros: LAYERS, FORMAT, TRANSPARENT, VERSION, SLD_BODY (opcional)

createWfsLayer(layerConfig)
// Crea una capa Vector con fetch manual a GeoServer WFS
// Formato: GeoJSON
// Proyección: EPSG:4326 → re proyecta a EPSG:3857

createBarriosLayer()
// WFS de pg_barriosurbanos
// Colores pasteles únicos por barrio (20 colores rotativos)

createUbaLayer(ubaId)
// WFS de pg_uba1..5 o pg_ubac
// Color pastel fijo por UBA:
//   UBA1: rosa, UBA2: verde, UBA3: azul cielo
//   UBA4: durazno, UBA5: violeta, UBAC: cian

```

Sincronización con contexto

Cada vez que `activeLayers` cambia en MapContext, MapViewer:

1. Compara el nuevo Set con el estado anterior
2. Capas agregadas: llama `createWmsLayer()` o `createWfsLayer()` según tipo
3. Capas removidas: busca la capa en `map.getLayers()` y la elimina

Click en el mapa - GetFeatureInfo

Orden de prioridad al hacer clic:

- 1. Alumbrado público (WFS: subestaciones, luminarias, rutas)
- 2. Barrios (WFS: pg_barriosurbanos)
- 3. UBAs (WFS: pg_uba1..5, pg_ubac)
- 4. Uso de suelo (WMS: pg_uds_*)
- 5. Equipo institucional (WFS)
- 6. Educación (WFS)
- 7. Sisben (WMS)
- 8. Predios urbanos (WMS: pg_predios_urbanos_m) — fallback

Para capas WFS: usa `forEachFeatureAtPixel()` con tolerancia de 10 píxeles. Para capas WMS: consulta `GetFeatureInfo` con `INFO_FORMAT=application/json` .

El popup muestra los atributos formateados con iconos según el tipo de capa.

5.6 LayerPanel (organisms/LayerPanel/LayerPanel.jsx)

Panel lateral con árbol de capas jerárquico de 3 niveles.

Estructura de grupos:

Grupo	Color	Subcategorías / Capas
Planeación	azul #1976d2	Catastro (Predios, Nomenclatura), UBAs (UBA1-5, UBAC, Barrios), Usos de Suelo (Estanco, Bares, Discotecas, Depósitos, Hospedaje, Parqueaderos)
Zonas Verdes	verde #10b981	Zonas Verdes
Infraestructura	ámbar #f59e0b	Alumbrado Público (Transformadores, Luminarias, Rutas), Pavimentación (Obras)

Interacciones:

- Clic en checkbox de capa → `toggleLayer(id)` en MapContext
- Clic en título de grupo → colapsa/expande grupo
- Clic en título de subcategoría → colapsa/expande subcategoría
- Badge en grupo muestra número de capas activas

5.7 MapToolbar (organisms/MapToolbar/MapToolbar.jsx)

Barra de herramientas SIG con los siguientes controles:

Herramienta	Acción en mapa
Medir distancia	Dibuja línea, muestra longitud en tiempo real
Medir área	Dibuja polígono, muestra área en m² o ha
Buffer	Clic genera círculo de influencia
Seleccionar por polígono	Dibuja polígono, lista features dentro

- Slider de radio de buffer: 10 m a 10,000 m (paso 10 m)
- Botón **Limpiar** (rojo) aparece solo cuando hay herramienta activa
- Botón **Buscar** abre el SearchPanel

5.8 StatsPanel (organisms/StatsPanel/StatsPanel.jsx)

Panel de estadísticas que se actualiza automáticamente cuando el usuario activa capas de UBAs o uso de suelo.

Secciones:

1. **Resumen por UBA** (aparece cuando hay ≥ 1 UBA activa)
 - Tabla con: UBA | Número de predios | Área en m²
 - Datos estáticos de UBA_DATA en config/layers.js
2. **Total por capa de uso de suelo** (cuando hay ≥ 1 capa de uso de suelo activa)
 - Muestra conteo total de establecimientos por tipo
 - Datos obtenidos de consultas WFS a GeoServer en tiempo real
3. **Establecimientos por UBA** (cuando hay UBAs + capas de uso de suelo)
 - Por cada UBA activa: barras horizontales con conteo por tipo de establecimiento
 - Porcentaje relativo al máximo

5.9 Configuración de GeoServer (config/geoserver.js)

```
GEOSERVER_CONFIG = {
  baseUrl: '/geoserver', // Via proxy Vite
  directUrl: 'http://localhost:8080/geoserver',
  workspace: 'AlcaldiaGeovisor',
  defaultCenter: [-76.483765, 3.012569], // Santander de Quilichao
  defaultZoom: 14,
  projections: {
    map: 'EPSG:3857',
    data: 'EPSG:4326'
  }
}
```

5.10 Definición de Capas (config/layers.js)

Archivo central de configuración con ~581 líneas. Define todas las capas organizadas por secretaría.

Estructura de una capa:

```
{
  id: 'predios_urbanos', // ID único interno
  name: 'Predios Urbanos', // Nombre de visualización
  type: 'wms', // 'wms' | 'wfs'
  geoserverLayer: 'pg_predios_urbanos_m', // Nombre en GeoServer
  workspace: 'AlcaldiaGeovisor',
  visible: false, // Visibilidad inicial
  queryable: true, // Habilita GetFeatureInfo
  opacity: 0.8,
```



```

sld: '...', // SLD (style) XML opcional para WMS
group: 'catastro' // Grupo lógico opcional
}

```

Capas configuradas por secretaría:

Secretaría	Capas
Planeación - Catastro	Pedios Urbanos (WMS), Nomenclatura Vial (WMS), Barrios Urbanos (WFS)
Planeación - UBAs	UBA 1, UBA 2, UBA 3, UBA 4, UBA 5, UBA C (todas WFS)
Planeación - Uso de suelo	Estanco, Bares, Discotecas, Depósitos Licores, Hospedaje, Parqueaderos (todas WMS)
Zonas Verdes	Zonas Verdes (WMS)
Infraestructura	Subestaciones, Luminarias LED, Rutas Alumbrado (WFS), Obras Pavimentación (WMS)

Datos estáticos de UBAs:

UBA	Pedios	Área m ²
UBA 1	2,197	718,315
UBA 2	5,959	1,662,210
UBA 3	2,805	1,054,055
UBA 4	3,258	902,978
UBA 5	1,537	608,186
UBA C	2,028	1,491,423

Funciones de utilidad:

```

getAllLayers() // Lista plana de todas las capas
getLayerById(id) // Capa por ID
getLayersBySecretaria(id) // Capas de una secretaría
getLayersByGroup(name) // Capas de un grupo
getWmsUrl(layer) // URL del endpoint WMS
getWfsUrl(layer, format) // URL del endpoint WFS (soporta GeoJSON)

```

5.11 Cliente HTTP (services/api.js)

Instancia de Axios configurada:

```

baseUrl: '/api'
timeout: 30000ms
headers: { 'Content-Type': 'application/json' }

```

Interceptores:

- **Request:** Agrega `Authorization: Bearer <token>` si existe en `localStorage`
- **Response:** Si status 401 → limpia token de `localStorage` y redirige a `/login`

Servicios exportados:

```
authService.login(username, password)
authService.logout()
authService.getProfile()
authService.refreshToken()

layersService.getAll()
layersService.getBySecretaria(secretariaId)
layersService.getLayerInfo(layerId)

statsService.getStatsByUba(ubaId)
statsService.getUsoSueloCount()
statsService.getGeneralStats()

formsService.submit(formData)
formsService.getHistory(secretariaId)
```

5.12 Servicio de Estadísticas (`services/statsService.js`)

Módulo que consulta GeoServer WFS directamente para calcular estadísticas dinámicas.

Flujo de `getStatsByUba(activeUbaLayers, activeUsoSueloLayers)` :

1. Para cada UBA activa:
 - a. Fetch WFS de la capa de barrios de esa UBA
→ obtiene Set de nombres de barrios
2. Para cada capa de uso de suelo activa:
 - a. Fetch WFS total count (conteo total de establecimientos)
 - b. Por cada barrio de cada UBA:
 - Fetch WFS con filtro CQL por nombre de barrio
 - Acumula conteo por UBA
3. Retorna:

```
{
  ubaData: { uba1: { predios, area }, ... },
  usoSueloCount: { uso_estanco: 145, uso_discotecas: 32, ... },
  porUba: { uba1: { uso_estanco: 23, ... }, uba2: {...}, ... }
}
```

6. Backend - Node.js + Express

6.1 Servidor (`server.js`)

```
// Configuración Express
app.use(cors({ origin: 'http://localhost:5173', credentials: true })))
app.use(rateLimit({ windowMs: 15 * 60 * 1000, max: 100 })))
app.use(express.json())

// Rutas
app.use('/api/auth', authRoutes)
app.use('/api/layers', layersRoutes)
app.use('/api/stats', statsRoutes)
app.use('/api/forms', formsRoutes)

// Health check
GET /api/health → { status: 'ok', timestamp }

// Puerto
PORT = process.env.PORT || 3001
```

6.2 Middleware de Autenticación (middleware/authMiddleware.js)

authMiddleware

- Extrae el token del header `Authorization: Bearer <token>`
- Verifica con `jwt.verify(token, JWT_SECRET)`
- Asigna `req.user = { id, username, role, secretaria }`
- Retorna 401 si el token falta, es inválido o expirado

roleMiddleware(requiredRoles[])

- Factory: retorna middleware que verifica `req.user.role`
- Retorna 403 si el rol no está en la lista

secretariaMiddleware(requiredSecretaria)

- `admin` y `editor_geo` pasan siempre
- Otros: `req.user.secretaria` debe coincidir con `requiredSecretaria`

6.3 Rutas de Autenticación (routes/auth.js)

Endpoint	Método	Auth	Descripción
/api/auth/login	POST	No	Valida credenciales, retorna JWT
/api/auth/me	GET	Sí	Retorna usuario autenticado
/api/auth/logout	POST	Sí	Mensaje de confirmación
/api/auth/refresh	POST	Sí	Genera nuevo JWT

Request de login:

```
{ "username": "admin", "password": "admin123" }
```

Response de login exitoso:

```
{
  "success": true,
  "token": "eyJhbGciOiJIUzI1NiIs... ",
  "user": {
    "id": 1,
    "username": "admin",
    "name": "Administrador",
    "role": "admin",
    "secretaria": null
  }
}
```

JWT:

- Secreto: `process.env.JWT_SECRET`
- Expiración: `process.env.JWT_EXPIRES_IN` (por defecto `'24h'`)
- Payload: `{ id, username, role, secretaria }`

6.4 Rutas de Capas (`routes/layers.js`)

Todos los endpoints requieren autenticación.

Endpoint	Método	Roles	Descripción
<code>/api/layers</code>	GET	todos	Lista de capas (filtrada por rol)
<code>/api/layers/secretaria/:id</code>	GET	todos	Capas de una secretaria
<code>/api/layers/:layerId</code>	GET	todos	Detalle de una capa

Usuarios con rol `admin` o `editor_geo` reciben todas las capas. Usuarios con rol `secretaria` o `lector` reciben solo las capas de su secretaria.

6.5 Rutas de Estadísticas (`routes/stats.js`)

Endpoint	Método	Descripción
<code>/api/stats/uba/:ubaId</code>	GET	Stats de una UBA (predios, área, población)
<code>/api/stats/uso-suelo</code>	GET	Conteo de establecimientos por tipo
<code>/api/stats/general</code>	GET	Totales generales del municipio

Actualmente los datos son estáticos (arrays en memoria). Pendiente: conexión a PostGIS.

6.6 Rutas de Formularios (`routes/forms.js`)

Endpoint	Método	Auth	Descripción
<code>/api/forms/submit</code>	POST	Sí	Envía formulario de datos
<code>/api/forms/history/:secretariaId</code>	GET	Sí	Historial de envíos

Restricciones:

- Rol lector no puede enviar formularios (solo lectura)
- secretaria : solo puede enviar/ver su propia secretaría
- admin y editor_geo : acceso completo

7. GeoServer - Capas WMS/WFS

7.1 Workspace

Todas las capas están publicadas en el workspace **AlcaldiaGeovisor** .

URL base: <http://localhost:8080/geoserver/AlcaldiaGeovisor>

7.2 Capas Publicadas

ID interno	Layer GeoServer	Tipo geométrico	Tipo servicio
predios_urbanos	pg_predios_urbanos_m	Polígono	WMS
nomenclatura_vial	SANTANDER IGAC 2025 — U_NOMENCLATURA_VIAL_2025	Línea	WMS
barrios_urbanos	pg_barriosurbanos	Polígono	WFS
uba1	pg_uba1	Polígono	WFS
uba2	pg_uba2	Polígono	WFS
uba3	pg_uba3	Polígono	WFS
uba4	pg_uba4	Polígono	WFS
uba5	pg_uba5	Polígono	WFS
ubac	pg_ubac	Polígono	WFS
uso_estanco	pg_uds_bar_estanco	Punto	WMS
uso_bares	pg_uds_bares	Punto	WMS
uso_discotecas	pg_uds_discos	Punto	WMS
uso_depositos	pg_uds_depositos_licores	Punto	WMS
uso_hospedaje	pg_uds_hospedaje	Punto	WMS
uso_parqueaderos	pg_uds_parqueaderos	Punto	WMS
zonas_verdes	pg_zonasverdes	Polígono	WMS
subestaciones	subestaciones_alumbradopublico	Punto	WFS
luminarias_led	pg_luminariasled_alumbradopublico	Punto	WFS
rutas_alumbrado	pg_rutas_alumbradopublico	Línea	WFS
obras_pavimentacion	obraspavimentacion_infraestructura	Polígono	WMS

7.3 Consultas WFS

Formato de URL de consulta WFS (ejemplo para barrios):

```
/geoserver/AlcaldiaGeovisor/ows
?service=WFS
&version=2.0.0
&request=GetFeature
&typeName=AlcaldiaGeovisor:pg_barriosurbanos
&outputFormat=application/json
&srsName=EPSG:4326
```

Con filtro CQL (ejemplo para contar por barrio):

```
&CQL_FILTER=nombre_barrio='EL CENTRO'
&resultType=hits
```

8. Autenticación y Roles

8.1 Flujo de Login

```
Usuario ingresa credenciales
→ POST /api/auth/login
→ Backend: bcrypt.compare(password, hash)
→ Si válido: jwt.sign({ id, username, role, secretaria }, secret, { expiresIn })
→ Frontend: localStorage.setItem('token', token)
→ AuthContext actualiza estado
→ Interceptor axios añade header en todas las peticiones
→ Redirige a /mapa
```

8.2 Protección de Rutas

Frontend (PrivateRoute):

```
// Si no autenticado → redirige a /login
// Puede verificar también un rol requerido
<PrivateRoute requiredRole="admin">
  <AdminPage />
</PrivateRoute>
```

Backend (middleware):

```
router.get('/protected', authMiddleware, roleMiddleware(['admin', 'editor_geo']), handler)
```

8.3 Roles del Sistema

Rol	Descripción	Acceso a capas	Operaciones
admin	Alcalde / Administrador TI	Todas	CRUD + gestión usuarios

editor_geo	Especialista GIS (Juan Pablo Holguín)	Todas	CRUD + gestión capas
secretaria	Jefe de secretaría	Solo su secretaría	Lectura + escritura propia
lector	Personal de apoyo	Solo su secretaría	Solo lectura

9. Flujos Principales

9.1 Carga de una Capa

1. Usuario marca checkbox en LayerPanel
2. toggleLayer(id) actualiza activeLayers en MapContext
3. MapViewer detecta cambio con useEffect
4. getLayerById(id) obtiene configuración de config/layers.js
5. Según type:
 - 'wms' → createWmsLayer() → ImageWMS source
 - 'wfs' → createWfsLayer() → fetch GeoJSON → VectorLayer
6. map.addLayer(olLayer)
7. Capa aparece en el mapa

9.2 Consulta de Atributos (Click)

1. Usuario hace clic en el mapa
2. MapViewer: handleMapClick(event)
3. Si hay herramienta activa → no procesar
4. forEachFeatureAtPixel() busca features WFS en el punto (10px tolerancia)
5. Según prioridad:
 - a. Feature WFS encontrado → formatear atributos
 - b. No encontrado → GetFeatureInfo WMS:


```
GET /geoserver/AlcaldiaGeovisor/wms?SERVICE=WMS&REQUEST=GetFeatureInfo
    &QUERY_LAYERS=...&INFO_FORMAT=application/json&I=...&J=...
```
6. Popup overlay muestra HTML formateado en las coordenadas del clic

9.3 Cálculo de Estadísticas

1. Usuario activa ≥ 1 UBA y ≥ 1 capa de uso de suelo
2. useMapStats hook detecta cambio en activeLayers
3. Llama statsService.getStatsByUba(activeUbas, activeUsos)
4. Por cada UBA activa:
 - Fetch WFS barrios de esa UBA
 - Por cada uso de suelo: fetch WFS count por barrio
5. StatsPanel re-renderiza con nuevos datos
 - Tabla resumen UBAs
 - Totales por tipo de establecimiento
 - Gráficos de barras por UBA

10. Herramientas SIG

Medir distancia

- Activa interacción `ol/interaction/Draw` de tipo `LineString`
- Muestra distancia acumulada en tiempo real junto al cursor
- Al completar: muestra distancia total formateada (m o km)

Medir área

- Activa interacción `Draw` de tipo `Polygon`
- Muestra área en tiempo real mientras se dibuja
- Al completar: muestra área en m² (< 10,000 m²) o ha (≥ 10,000 m²)

Buffer

- Clic en el mapa genera un círculo
- Radio configurable: 10 m a 10,000 m (slider en MapToolbar)
- Se visualiza como polígono de influencia en el mapa

Seleccionar por polígono

- Activa interacción `Draw` de tipo `Polygon`
- Al completar: busca todas las features WFS activas dentro del polígono
- Resultados mostrados en `SelectionResults` (panel lateral)

Limpiar

- Elimina todas las geometrías de dibujo del mapa
- Desactiva la herramienta activa
- Limpia `selectionResults`

11. Panel de Estadísticas

El `StatsPanel` se alimenta del hook `useMapStats`, que a su vez usa `statsService`.

Datos de UBAs (estáticos)

Estos datos están definidos en `config/layers.js` como `UBA_DATA`:

UBA	Pedios	Área m ²
UBA 1	2,197	718,315
UBA 2	5,959	1,662,210
UBA 3	2,805	1,054,055
UBA 4	3,258	902,978
UBA 5	1,537	608,186
UBA C	2,028	1,491,423

Conteos dinámicos (consulta WFS en tiempo real)

Para las capas de uso de suelo se consulta directamente GeoServer con `resultType=hits` para obtener conteos eficientes sin transferir geometrías.

12. Configuración y Despliegue

Variables de Entorno

Backend (`.env` en `backend/`):

```
PORT=3001
JWT_SECRET=tu_secreto_seguro_aqui
JWT_EXPIRES_IN=24h
NODE_ENV=development
```

Iniciar en Desarrollo

```
# Terminal 1 - Backend
cd alcaldia-geovisor/backend
npm install
npm run dev
# Servidor en http://localhost:3001

# Terminal 2 - Frontend
cd alcaldia-geovisor/frontend
npm install
npm run dev
# App en http://localhost:5173

# GeoServer debe estar corriendo en http://localhost:8080
```

Build de Producción

```
cd alcaldia-geovisor/frontend
npm run build
# Genera dist/ con archivos estáticos
```

Dependencias del Sistema

- Node.js >= 18
- GeoServer >= 2.23 con workspace `AlcaldiaGeovisor` configurado
- PostGIS (base de datos de GeoServer)

13. Usuarios de Desarrollo

Los usuarios son actualmente mock (arrays en memoria en `backend/src/routes/auth.js`):

Usuario	Contraseña	Rol	Secretaría	Descripción
admin	admin123	admin	—	Acceso total al sistema
juanpablo	geovisor123	editor_geo	—	Juan Pablo Holguín, gestión de capas

planeacion	planeacion123	secretaria	planeacion	Jefe de Planeación Municipal
educacion	educacion123	secretaria	educacion	Jefe de Educación Municipal
salud	salud123	secretaria	salud	Jefe de Salud Municipal
lector	lector123	lector	planeacion	Usuario de solo lectura

14. Estado Actual del Proyecto

Completado

- Estructura completa de carpetas (frontend + backend)
- Configuración de Vite con proxy para GeoServer y API
- Sistema de autenticación JWT (login, logout, PrivateRoute, interceptores)
- MapContext con estado global del mapa OpenLayers
- MapViewer completo con:
 - Creación de capas WMS y WFS
 - GetFeatureInfo con prioridades
 - Herramientas SIG (medir, buffer, seleccionar)
 - Colores pasteles por UBA y barrio
- LayerPanel con árbol jerárquico de 3 niveles
- MapToolbar con controles de herramientas
- StatsPanel con estadísticas dinámicas por UBA
- Panel de Header con logos y navegación
- Páginas: Login, Mapa, Dashboard
- Backend Express con todos los endpoints
- Servicio de estadísticas WFS en tiempo real
- Build de producción funcional (dist/)

Pendiente / En desarrollo

- Conexión a base de datos real (actualmente mock en memoria)
- SearchPanel: búsqueda por dirección o predio
- SelectionResults: listado completo de features seleccionados
- Gestión de usuarios desde la interfaz (solo admin)
- Módulo de formularios de campo (secretarías)
- Despliegue en servidor de producción
- Integración con más secretarías (salud, gobierno, etc.)